

Experiment No. 3

Name: Parth Vijay Gulhane

Roll No: 23CO043

Class: TE Comp A

Title

Implement Greedy search algorithm for any of the following application:

- I. Selection Sort
- II. Minimum Spanning Tree
- III. Single-Source Shortest Path Problem
- IV. Job Scheduling Problem
- V. Prim's Minimal Spanning Tree Algorithm
- VI. Kruskal's Minimal Spanning Tree Algorithm
- VII. Dijkstra's Minimal Spanning Tree Algorithm

Theory

1. Selection Sort (Greedy Approach)

Selection Sort is a simple sorting algorithm that repeatedly selects the minimum element from the unsorted portion and places it at the correct position.

Greedy Nature

At each step:

- Choose the smallest available element
- Place it immediately in final position

Steps

1. Find minimum element
2. Swap with first position
3. Move boundary forward
4. Repeat

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

2. Minimum Spanning Tree (MST)

A Minimum Spanning Tree is a subset of edges in a graph that:

- Connects all vertices
- Has no cycles
- Has minimum total weight

Greedy Nature

Edges are selected based on minimum weight while avoiding cycles.

Applications

- Network design
- Road/rail systems
- Cable connections

3. Single Source Shortest Path

Find the shortest path from one source node to all other nodes in a weighted graph.

Greedy Approach

Use Dijkstra's Algorithm:

- Always pick the nearest node
- Update distances

Important

- Works only for non-negative weights

4. Job Scheduling Problem

Given jobs with:

- Deadline
- Profit

Schedule jobs to maximize total profit within deadlines.

Greedy Strategy

- Sort jobs by maximum profit
- Assign each job to the latest available slot

Applications

- CPU scheduling
- Task management

5. Prim's Algorithm (MST)

Prim's Algorithm constructs MST by:

- Starting from a node
- Expanding tree by adding minimum weight edge

Greedy Choice

Pick smallest edge connecting tree to new vertex

Steps

1. Start with any node
2. Select minimum edge
3. Add to MST
4. Repeat

6. Kruskal's Algorithm (MST)

Kruskal's Algorithm builds MST by:

- Sorting all edges
- Adding smallest edge without forming cycle

Greedy Choice

Always pick smallest edge globally

Key Concept

- Uses Union-Find to detect cycles

7. Dijkstra's Algorithm

Dijkstra's Algorithm finds shortest path from a source node to all other nodes.

Greedy Choice

Select node with minimum distance

Key Idea

Once a node is finalized → its distance is fixed

COMPARISON TABLE

Algorithm	Problem Type	Greedy Choice
Selection Sort	Sorting	Pick minimum element
MST	Graph	Pick smallest edge
Dijkstra	Shortest Path	Pick nearest node
Job Scheduling	Optimization	Pick highest profit job
Prim's	MST	Expand with min edge
Kruskal's	MST	Global min edge
Dijkstra	Shortest Path	Local min distance

Code :

```
#include <iostream>
#include <vector>
#include <queue>
#include <limits>
#include <utility>
using namespace std;

void dijkstra(vector<vector<pair<int,int>>> &graph, int start) {
    int n = graph.size();
    vector<int> distances(n, numeric_limits<int>::max());
    distances[start] = 0;
    // Min-heap → (distance, node)
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0, start});
    int step = 1;
    while (!pq.empty()) {
        cout << "\n--- Step " << step << " ---\n";
```

```

// Print PQ
cout << "Priority Queue: ";
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> temp = pq;
while (!temp.empty()) {
    cout << "(" << temp.top().first << ", " << temp.top().second << ") ";
    temp.pop();
}
cout << endl;
// Print distances
cout << "Distances: ";
for (int i = 0; i < n; i++) {
    if (distances[i] == numeric_limits<int>::max())
        cout << "inf ";
    else
        cout << distances[i] << " ";
}
cout << endl;
int current_distance = pq.top().first;
int current_node = pq.top().second;
pq.pop();
cout << "Selected Node: " << current_node
    << " with distance " << current_distance << endl;
if (current_distance > distances[current_node]) {
    cout << "Skipped (better path already exists)\n";
    continue;
}
for (auto &edge : graph[current_node]) {
    int neighbor = edge.first;
    int weight = edge.second;
    int new_distance = current_distance + weight;
    cout << "Checking edge " << current_node << " -> "
        << neighbor << " with weight " << weight << endl;
    if (new_distance < distances[neighbor]) {
        cout << "Updating distance of node " << neighbor
            << " from ";
        if (distances[neighbor] == numeric_limits<int>::max())
            cout << "inf";
        else
            cout << distances[neighbor];
        cout << " to " << new_distance << endl;
        distances[neighbor] = new_distance;
        pq.push({new_distance, neighbor});
    } else {
        cout << "No update needed for node " << neighbor << endl;
    }
}
}
step++;
}
// Final result
cout << "\nFinal Shortest Distances:\n";

```

```

for (int i = 0; i < n; i++) {
    cout << i << " -> ";
    if (distances[i] == numeric_limits<int>::max())
        cout << "inf";
    else
        cout << distances[i];
    cout << endl;
}
}

int main() {
    int n, e;
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    vector<vector<pair<int,int>>> graph(n);
    cout << "Enter edges (u v w):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        graph[u].push_back({v, w});
        graph[v].push_back({u, w}); // remove if directed
    }
    int start;
    cout << "Enter source node: ";
    cin >> start;
    dijkstra(graph, start);
    return 0;
}

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 3> cd "C:\Users\parth\Desktop\AI\Assignment\Assignment 3\" ; if ($?) { g++ A3.cpp -o A3 } ; if ($?) { .\A3 }
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v w):
0 1 4
0 2 1
1 2 2
1 3 5
2 3 8
Enter source node: 0

--- Step 1 ---
Priority Queue: (0,0)
Distances: 0 inf inf inf
Selected Node: 0 with distance 0
Checking edge 0 -> 1 with weight 4
Updating distance of node 1 from inf to 4
Checking edge 0 -> 2 with weight 1
Updating distance of node 2 from inf to 1

--- Step 2 ---
Priority Queue: (1,2) (4,1)
Distances: 0 4 1 inf
Selected Node: 2 with distance 1
Checking edge 2 -> 0 with weight 1
No update needed for node 0
Checking edge 2 -> 1 with weight 2
Updating distance of node 1 from 4 to 3
Checking edge 2 -> 3 with weight 8
Updating distance of node 3 from inf to 9

--- Step 3 ---
Priority Queue: (3,1) (4,1) (9,3)
Distances: 0 3 1 9
Selected Node: 1 with distance 3
Checking edge 1 -> 0 with weight 4
No update needed for node 0

```

```
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 3> cd "c:\Users\parth\Desktop\AI\Assignment\Assignment 3\" ; if ($?) { g++ A3.cpp -o A3 } ; if ($?) { .\A3 }
Distances: 0 3 1 9
Selected Node: 1 with distance 3
Checking edge 1 -> 0 with weight 4
No update needed for node 0
Checking edge 1 -> 2 with weight 2
No update needed for node 2
Checking edge 1 -> 3 with weight 5
Updating distance of node 3 from 9 to 8

--- Step 4 ---
Priority Queue: (4,1) (8,3) (9,3)
Distances: 0 3 1 8
Selected Node: 1 with distance 4
Skipped (better path already exists)

--- Step 4 ---
Priority Queue: (8,3) (9,3)
Distances: 0 3 1 8
Selected Node: 3 with distance 8
Checking edge 3 -> 1 with weight 5
No update needed for node 1
Checking edge 3 -> 2 with weight 8
No update needed for node 2

--- Step 5 ---
Priority Queue: (9,3)
Distances: 0 3 1 8
Selected Node: 3 with distance 9
Skipped (better path already exists)

Final Shortest Distances:
0 -> 0
1 -> 3
2 -> 1
3 -> 8
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 3> |
```